Generated: `2026-03-15 01:19:08`

ExtendableHashTableBenchmark

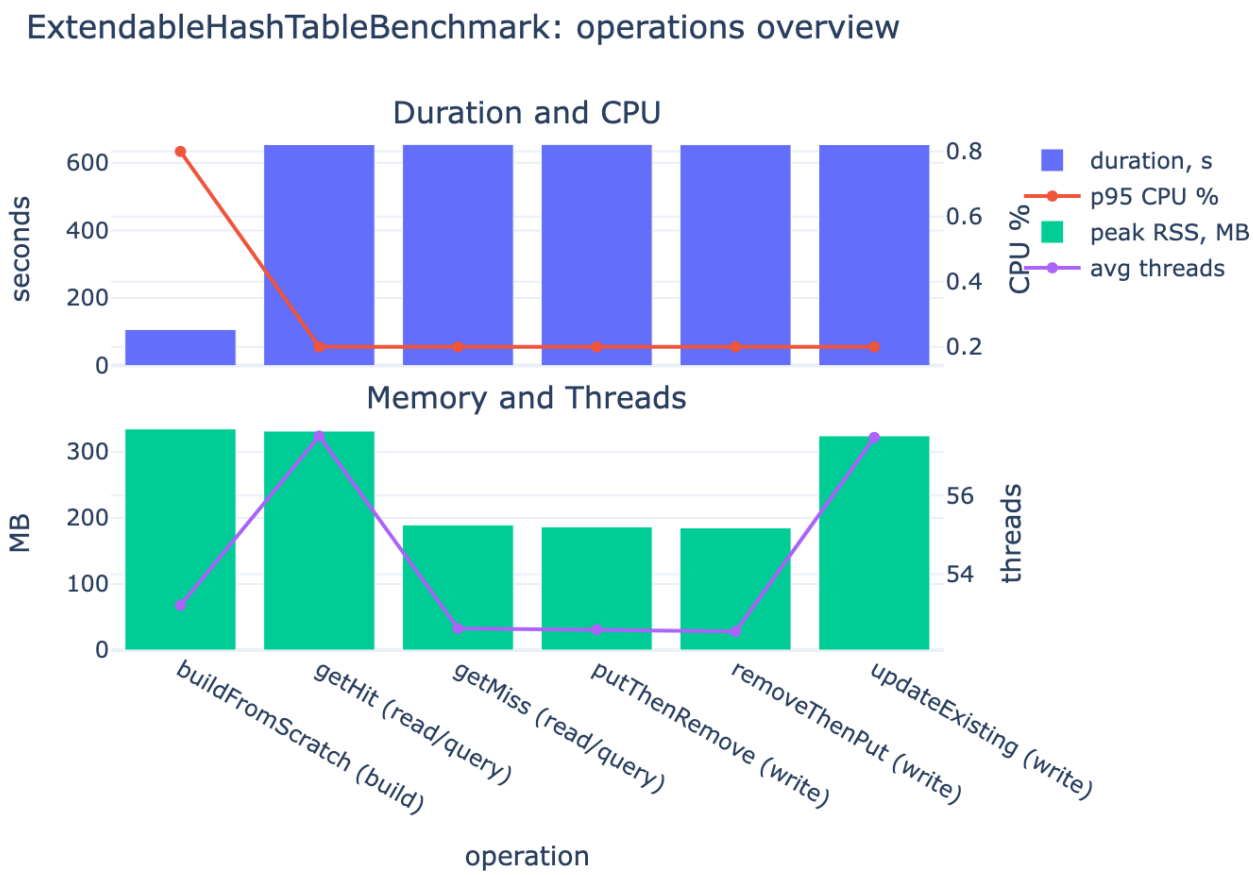

Figure 1: overview

operation	kind	duration_s	p95_cpu_%	peak_rss_mb	score_median
buildFromScratch	build	106.776	0.800	334.703	413.194
getHit	read/query	653.918	0.200	331.516	26.637
getMiss	read/query	654.388	0.200	189.484	35.872
putThenRemove	write	654.354	0.200	186.562	16.956
removeThenPut	write	653.696	0.200	184.953	13.758
updateExisting	write	653.978	0.200	324.344	16.016

buildFromScratch

- Operation type: `build`
- Parameters:
  - `bucketCapacity`: 8, 16
  - n: 200000, 350000
  - `seed`: 42

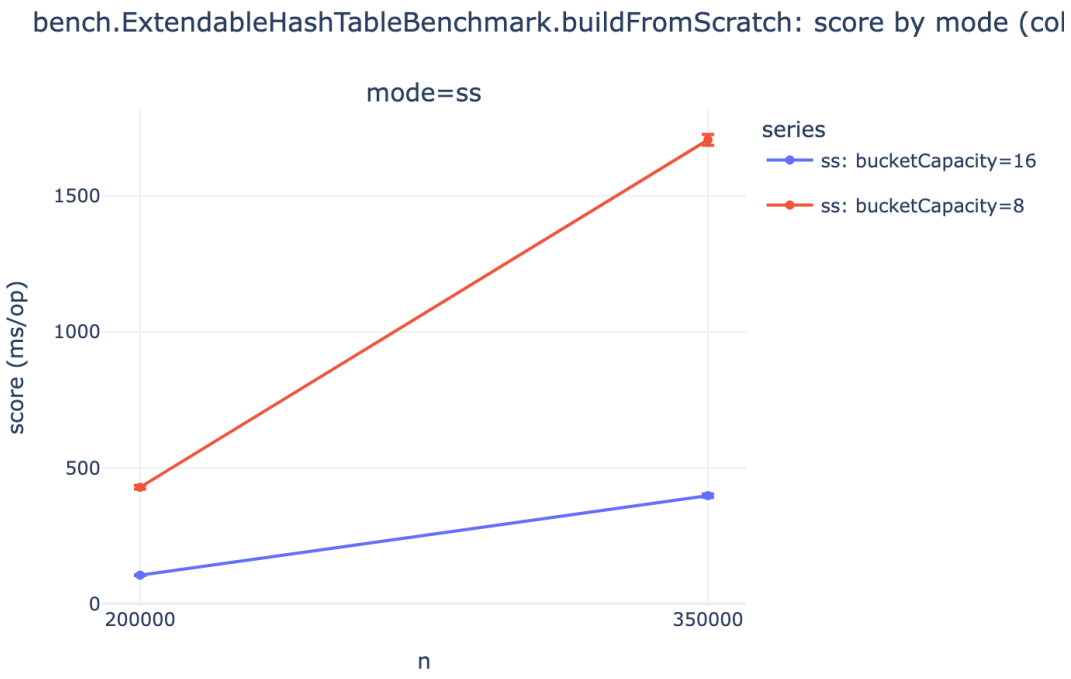

Figure 2: score

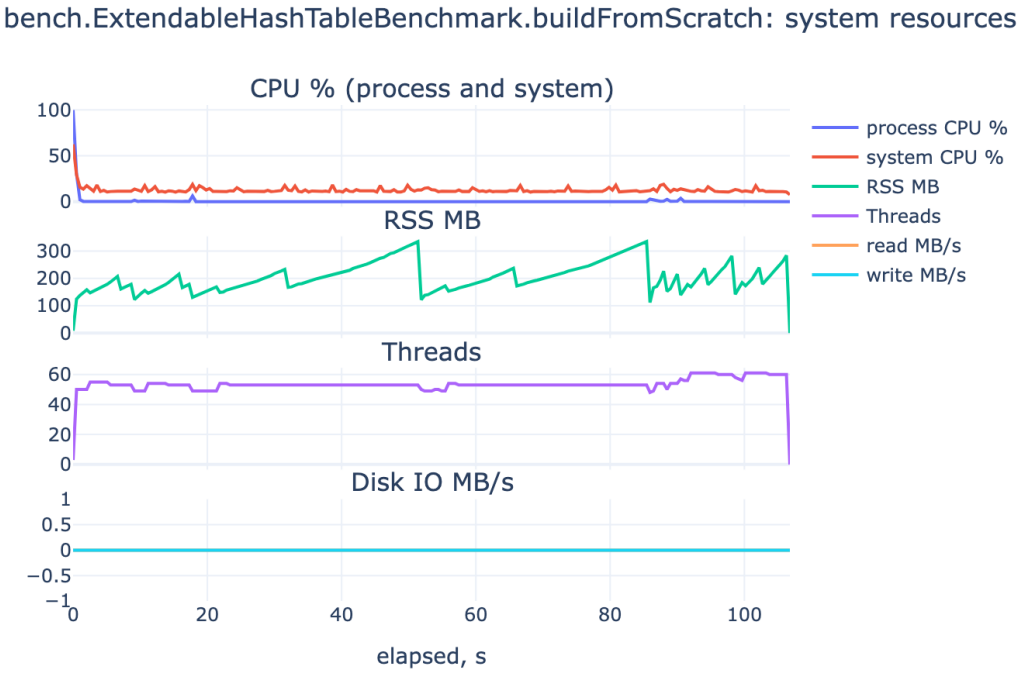

Figure 3: resources

Notes

- Построение самое тяжелое в этом алгоритме, median ~413 ms/op, сильный разброс по параметрам n, buck–

etCapacity.

- Фикс: заранее резервировать структуру бакетов/директории и минимизировать перевыделения при split, переиспользовать рабочие буферы чтения/записи.

getHit

- Operation type: `read/query`
- Parameters:
  - `bucketCapacity`: 8, 16
  - n: 200000, 350000
  - `seed`: 42

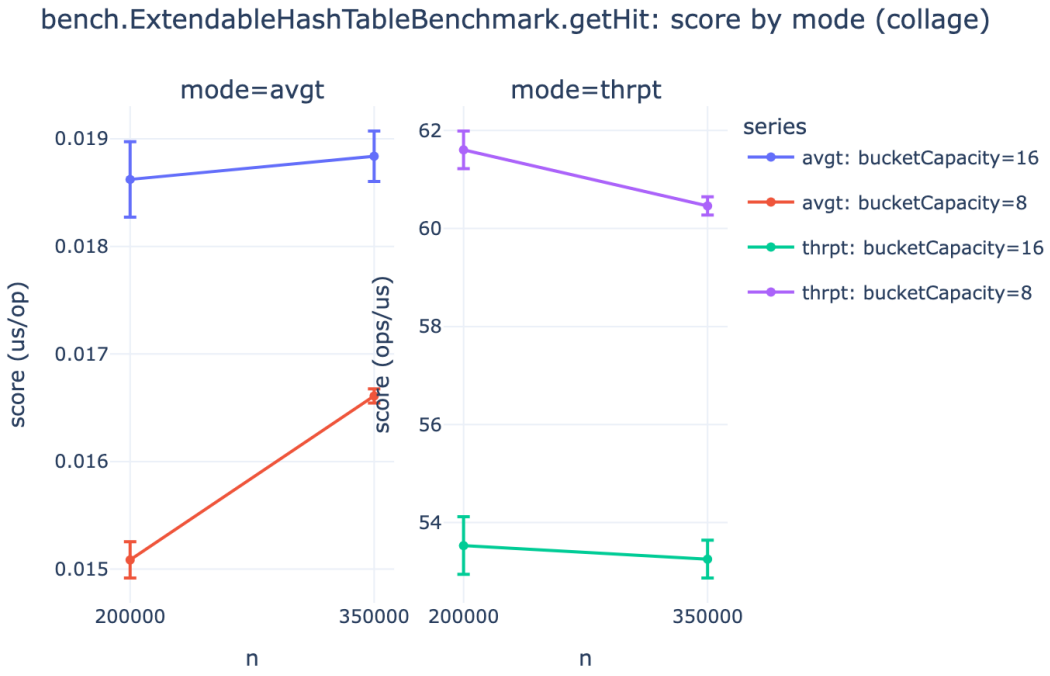

Figure 4: score

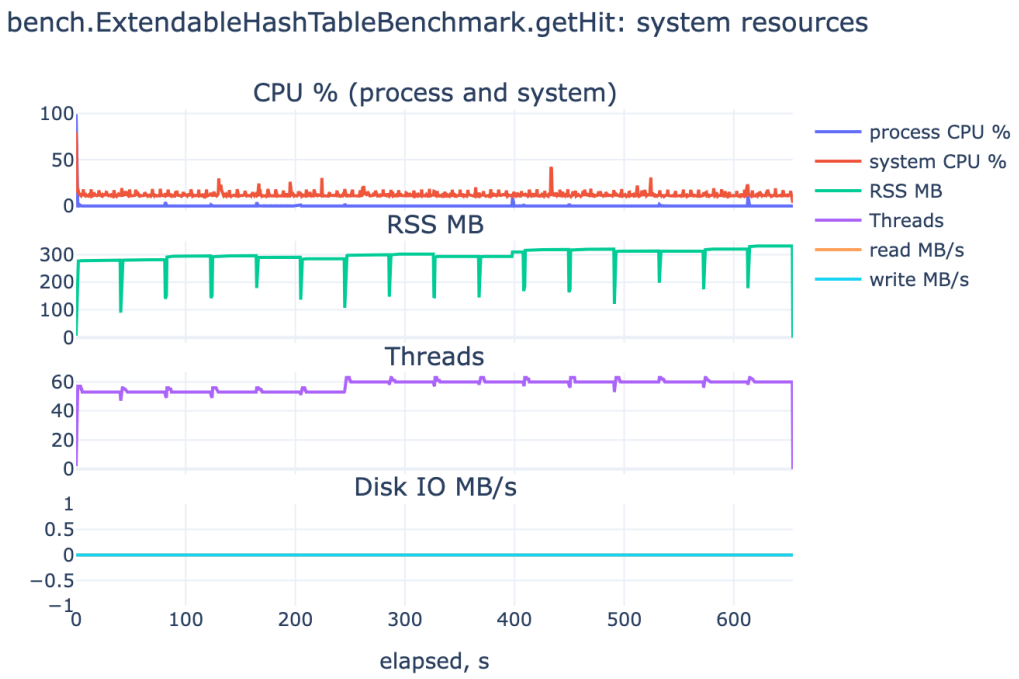

Figure 5: resources

Notes

- Чтение hit стабильно быстрое.
- Фикс: кэш горячих бакетов в state бенчмарка для лучшей locality, убрать лишние чтения метаданных бакета в hot-path get.

getMiss

- Operation type: `read/query`
- Parameters:
  - `bucketCapacity`: 8, 16
  - n: 200000, 350000
  - `seed`: 42

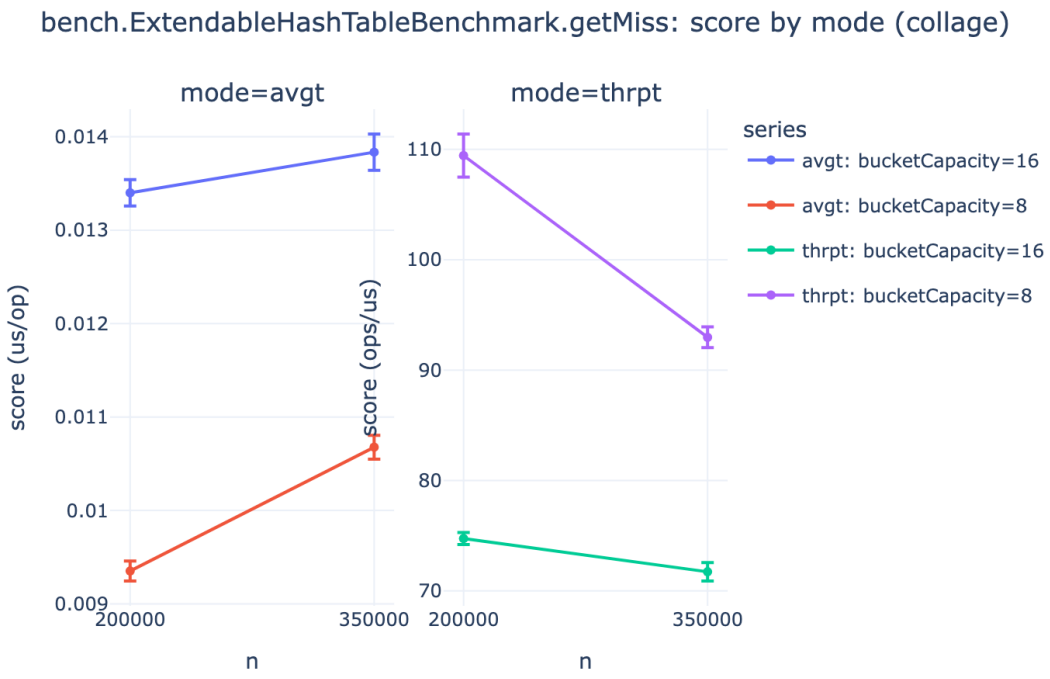

Figure 6: score

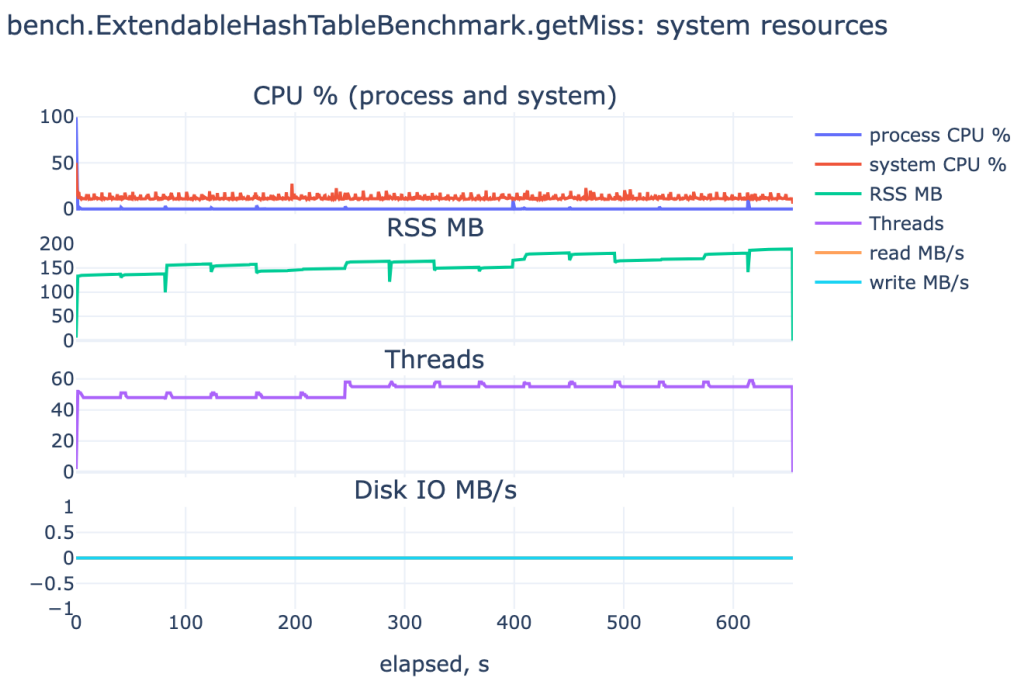

Figure 7: resources

Notes

- Miss быстрее hit, что логично, так как раньше завершается без чтения значения.

putThenRemove

- Operation type: `write`
- Parameters:
  - `bucketCapacity`: 8, 16
  - n: 100000, 200000
  - `seed`: 42

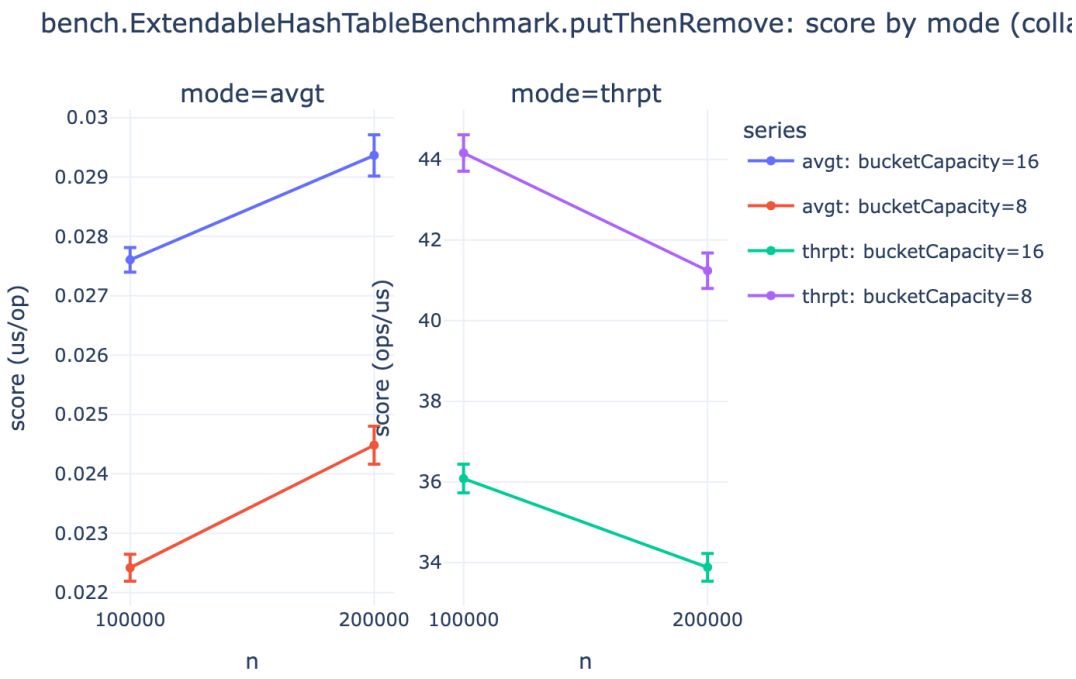

Figure 8: score

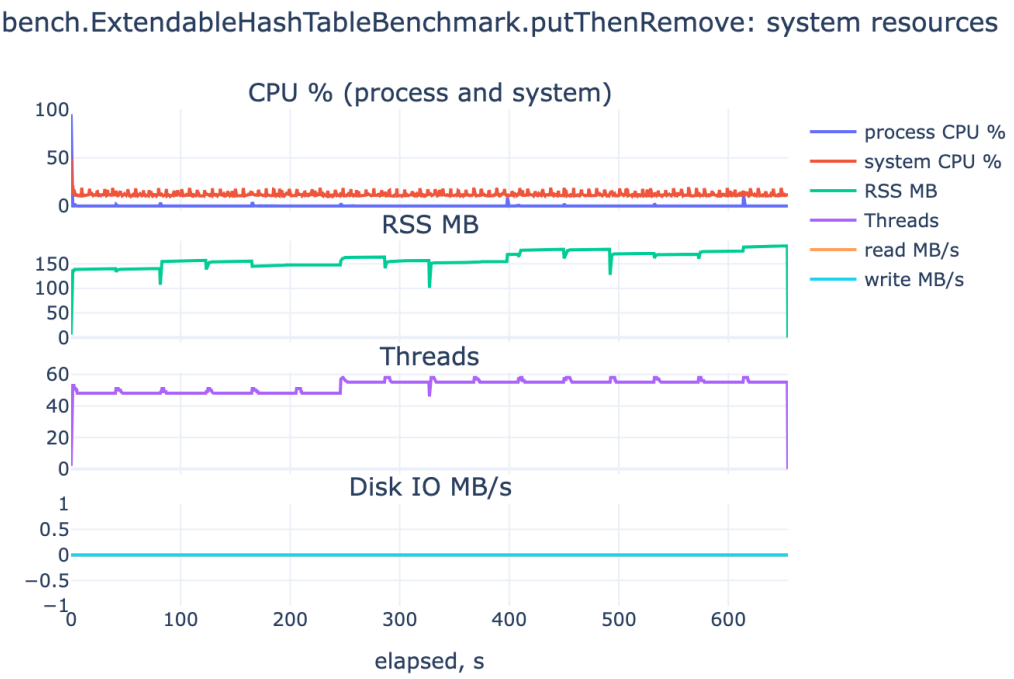

Figure 9: resources

Notes

- Запись+удаление: avgt ~0.0260 us/op, thrpt ~38.7 ops/us.
- Фикс: объединить серию мелких записей в один проход, переиспользовать tombstone-слоты перед split бакета.

removeThenPut

- Operation type: `write`
- Parameters:
  - `bucketCapacity`: 8, 16
  - n: 100000, 200000
  - `seed`: 42

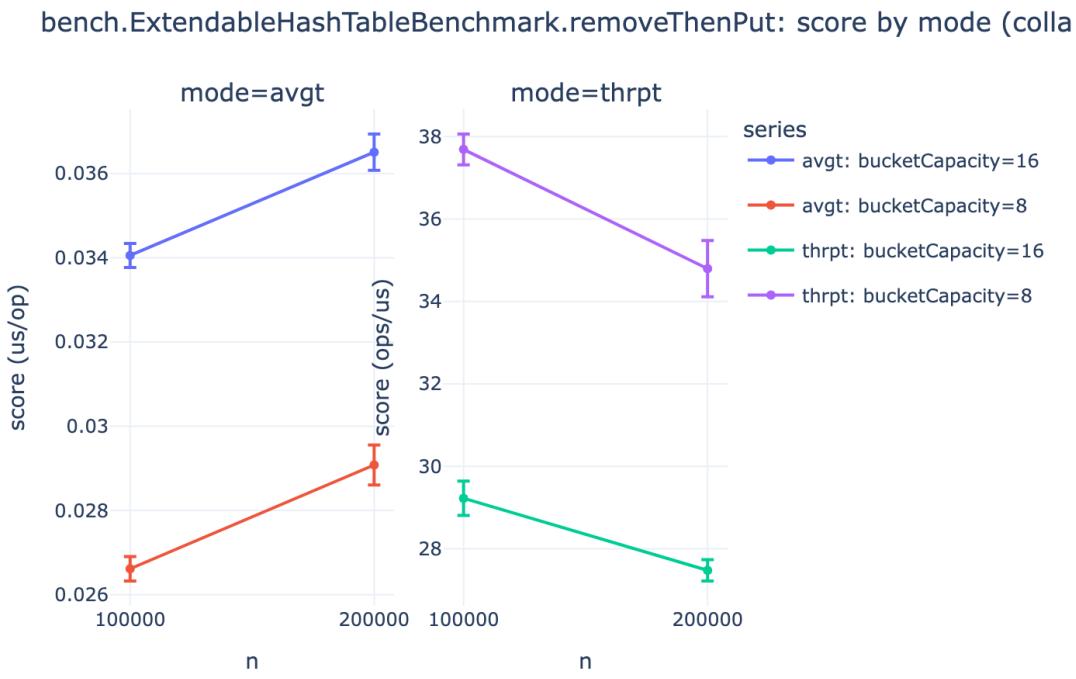

Figure 10: score

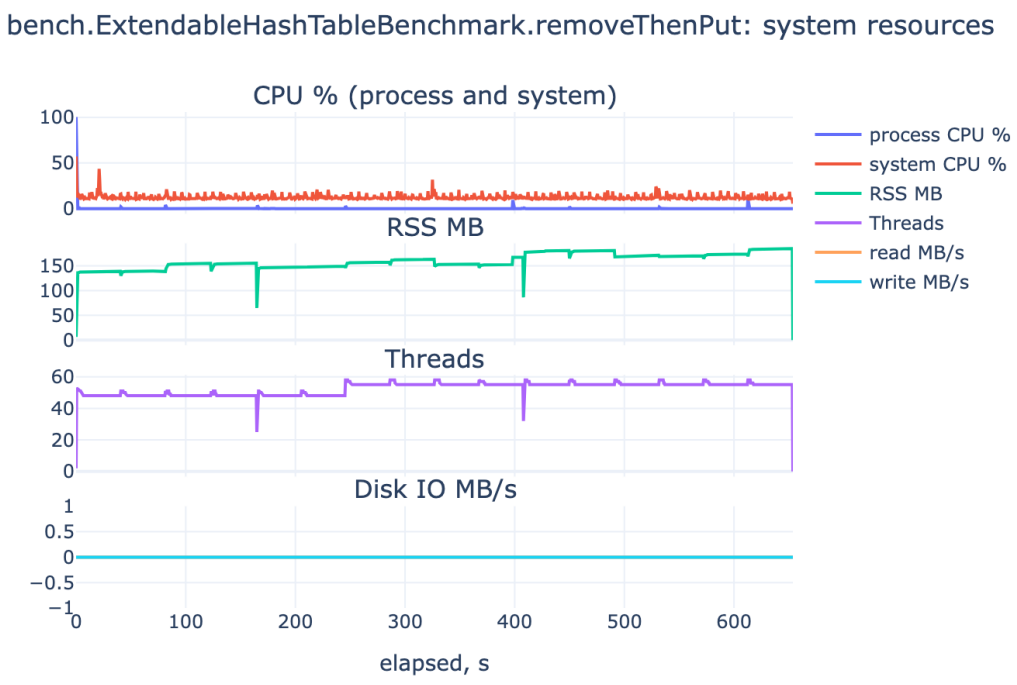

Figure 11: resources

Notes

- Сценарий remove->put медленнее put->remove.

- Фикс: после remove сразу оставлять slot в состоянии, удобном для последующего put без повторного скана.

updateExisting

- Operation type: `write`
- Parameters:
  - `bucketCapacity`: 8, 16
  - n: 100000, 200000
  - `seed`: 42

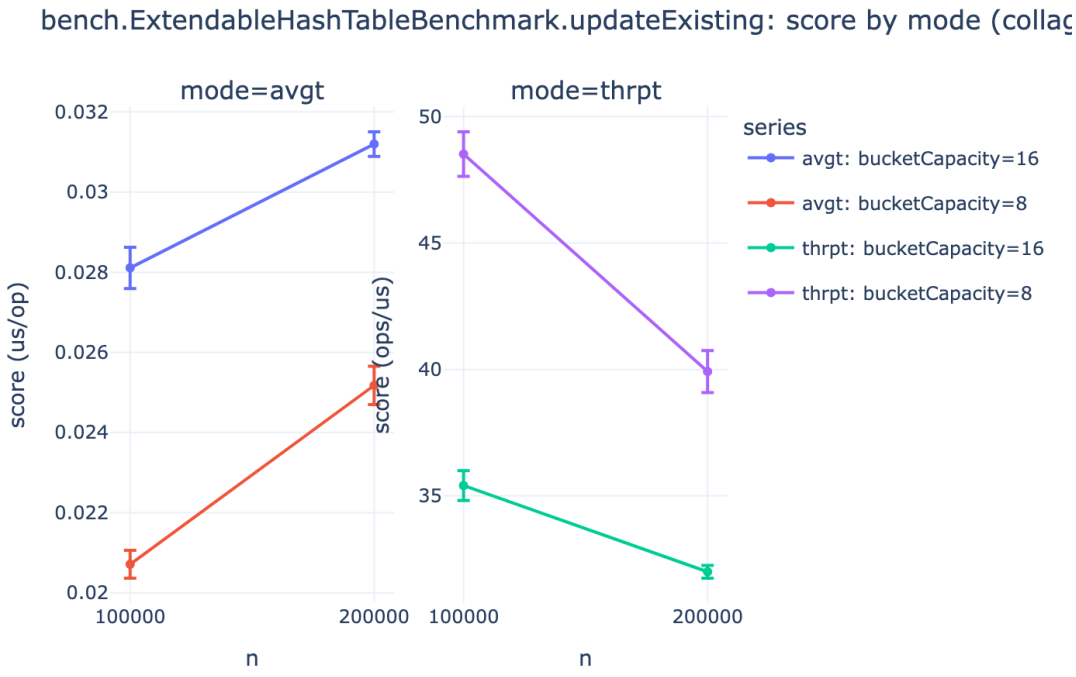

Figure 12: score

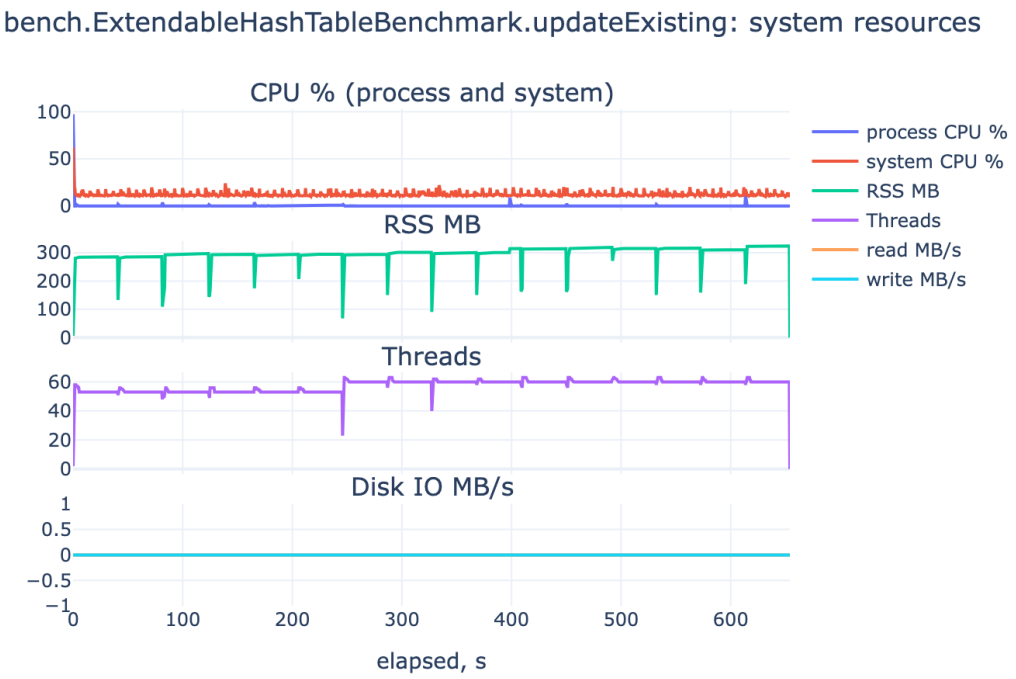

Figure 13: resources

Notes

- Update близок к другим write-операциям.
- Фикс: если новое значение равно старому, выходить без физической записи.

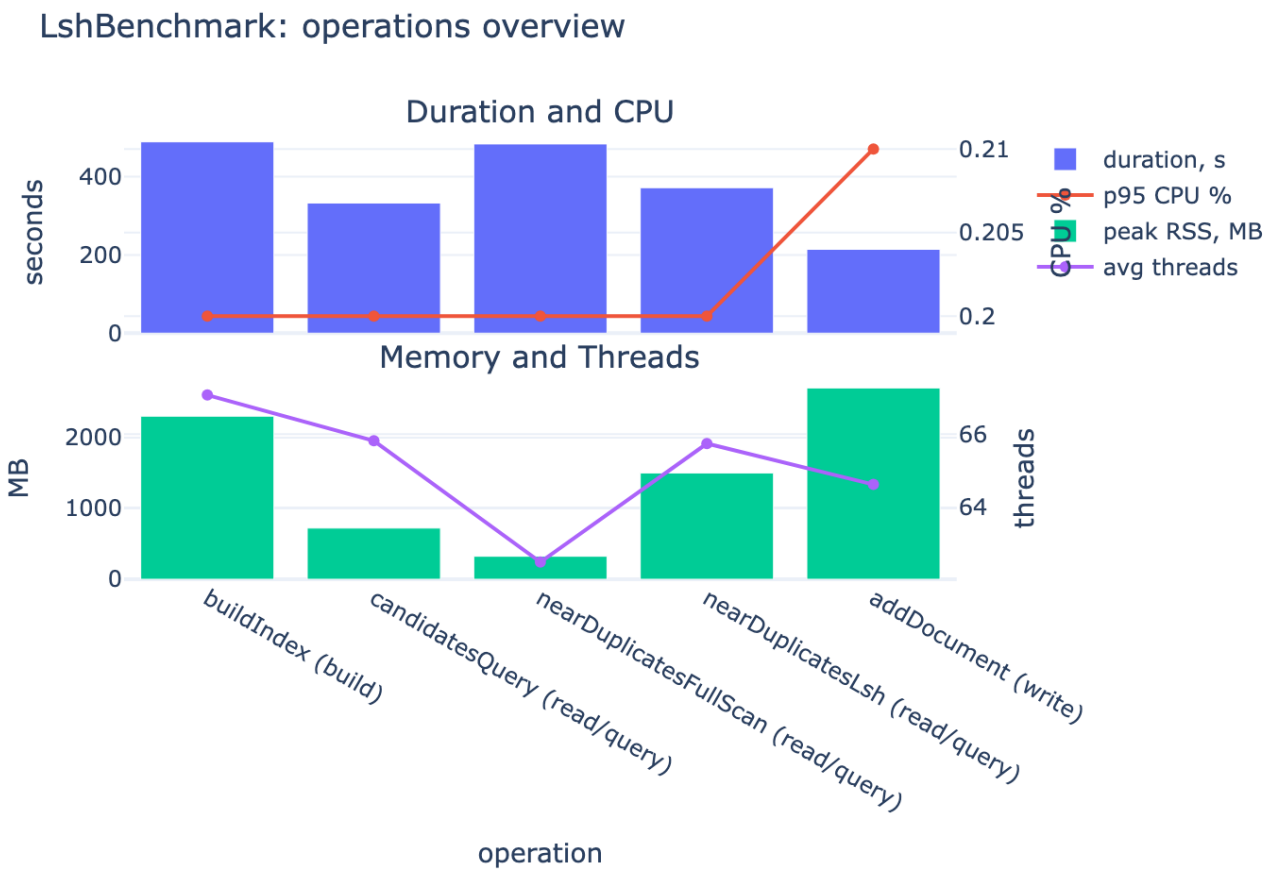

Figure 14: overview

operation	kind	duration_s	p95_cpu_%	peak_rss_mb	score_median
buildIndex	build	488.716	0.200	2304.516	307.792
candidatesQuery	read/query	332.474	0.200	723.000	13.563
nearDup FullScan	read/query	483.371	0.200	324.719	425.400
nearDup Lsh	read/query	371.598	0.200	1502.062	9.224
addDocument	write	214.170	0.210	2702.844	14.595

addDocument

- Operation type: `write`
- Parameters:
  - `baseDocs`: 20000
  - `seed`: 42
  - `wordsPerDoc`: 16

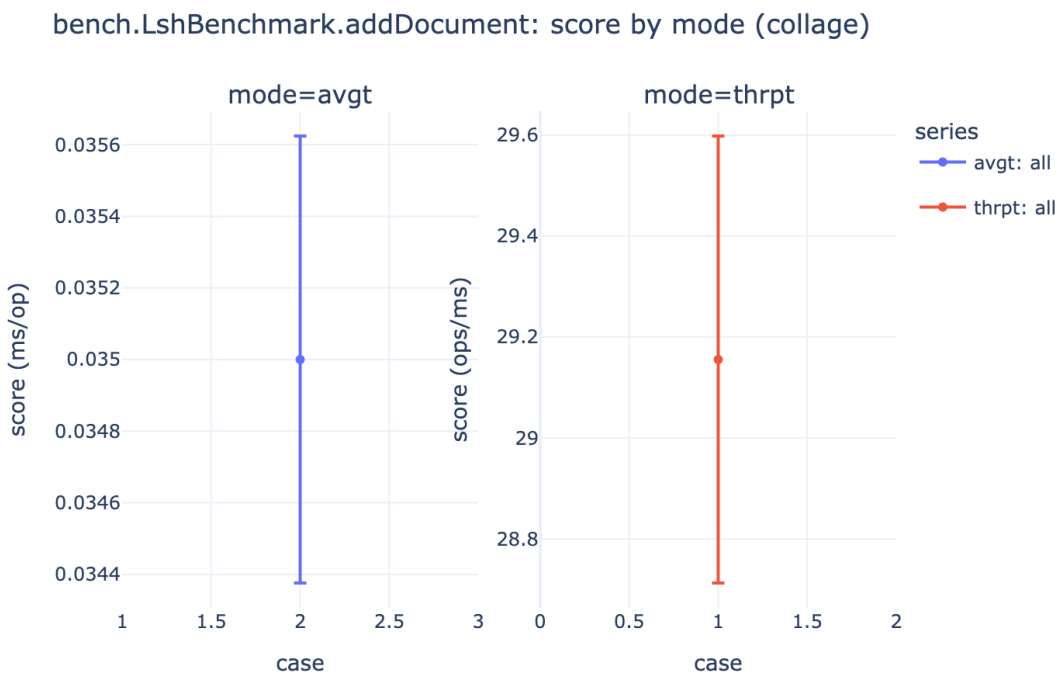

Figure 15: score

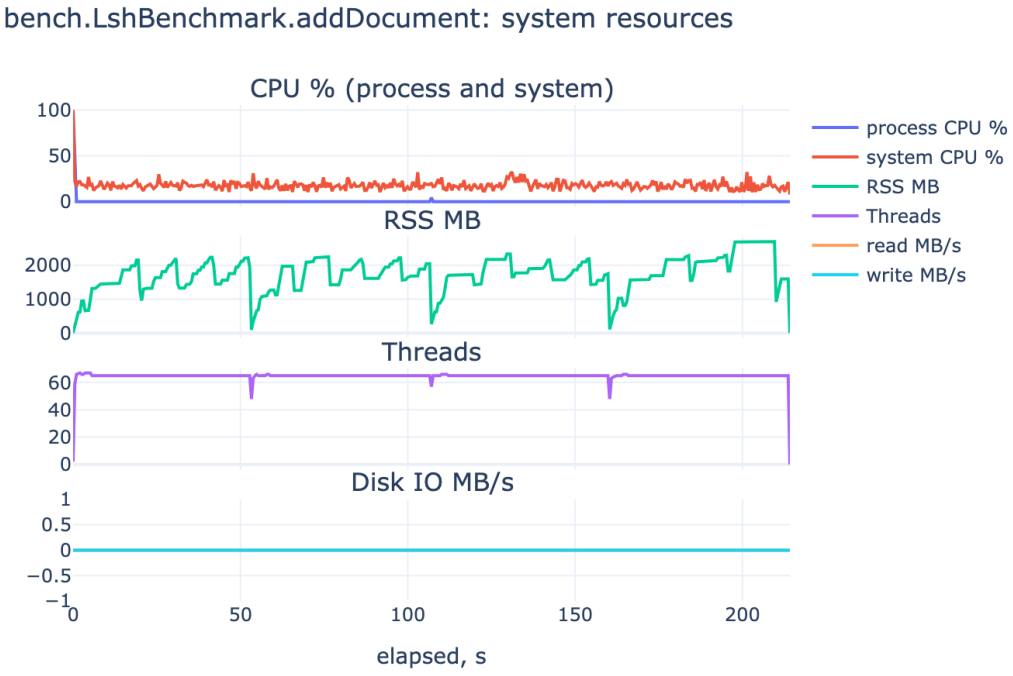

Figure 16: resources

Notes

- Добавление документа быстрое, но общиее использование памяти большое на больших данных.

- Фикс: переиспользовать временные массивы/хеши для шинглов между вызовами.

buildIndex

- Operation type: `build`
- Parameters:
  - `docs`: 20000, 60000
  - `seed`: 42
  - `wordsPerDoc`: 16

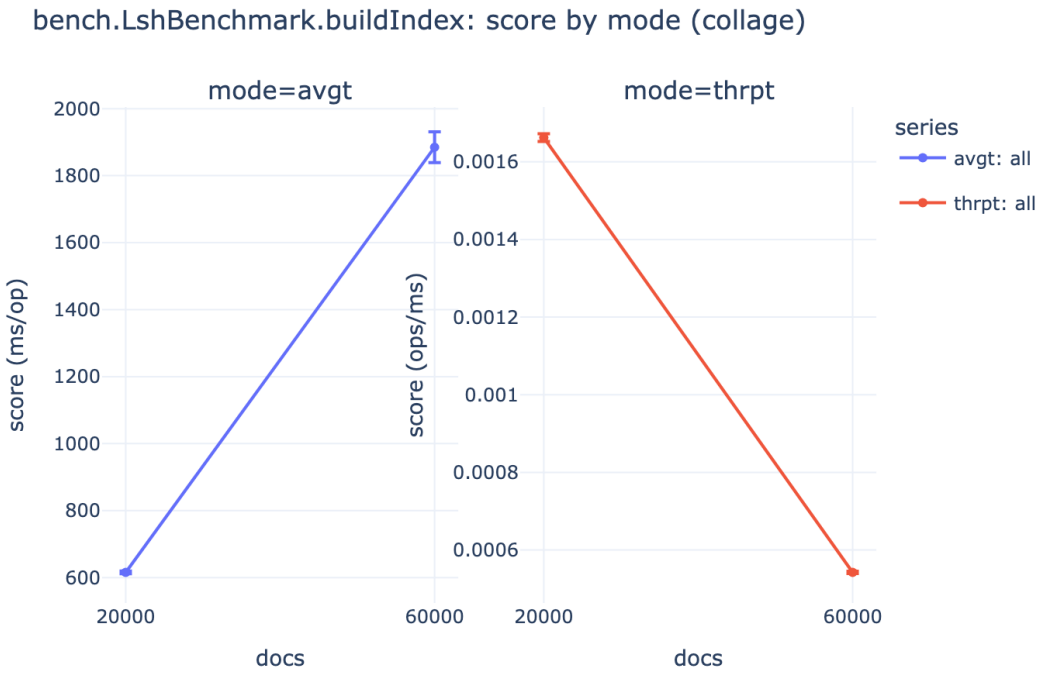

Figure 17: score

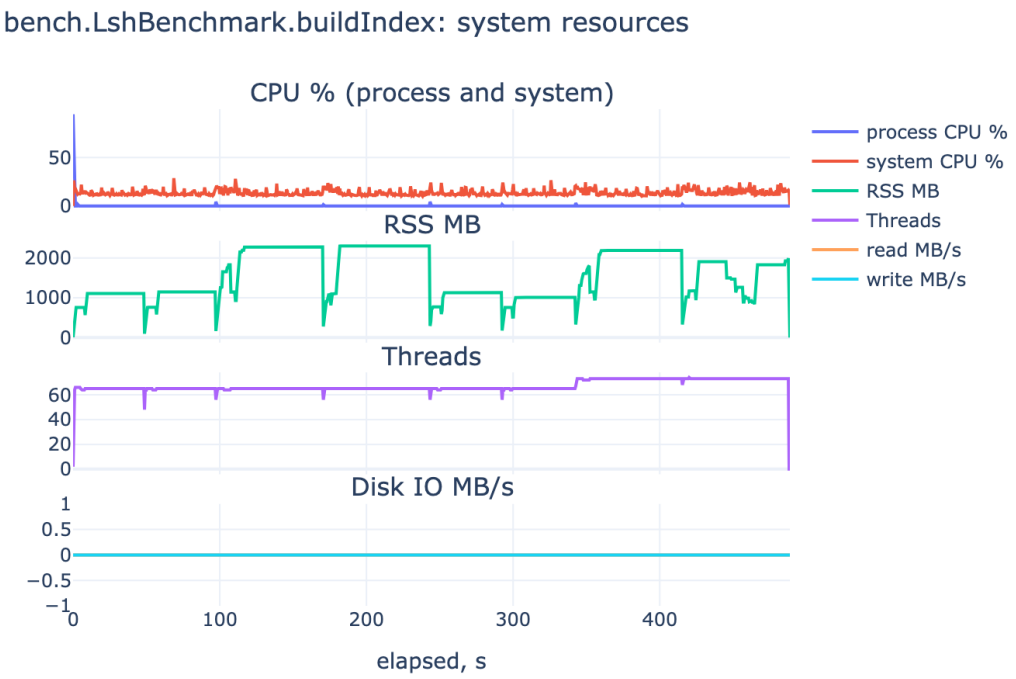

Figure 18: resources

Notes

- Самая тяжелая стадия LSH: avgt median ~1250 ms/op, пик памяти ~2.3 GB.
- Фикс: перейти на примитивные коллекции/массивы для сигнатур.

candidatesQuery

- Operation type: `read/query`
- Parameters:
  - `docs`: 20000, 60000
  - `seed`: 42
  - `wordsPerDoc`: 16

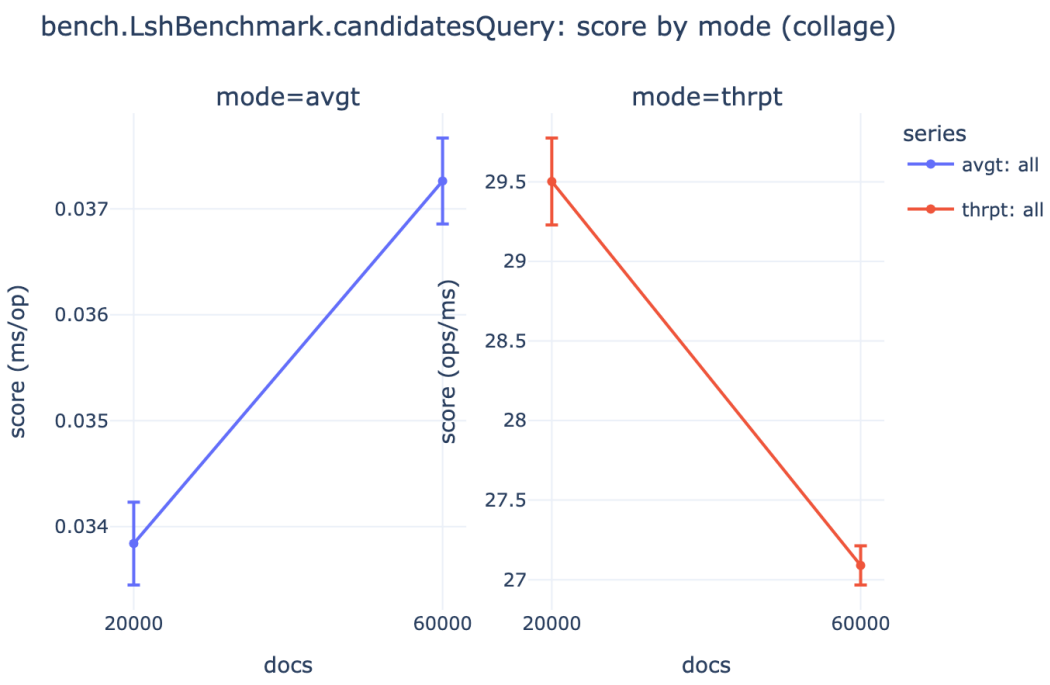

Figure 19: score

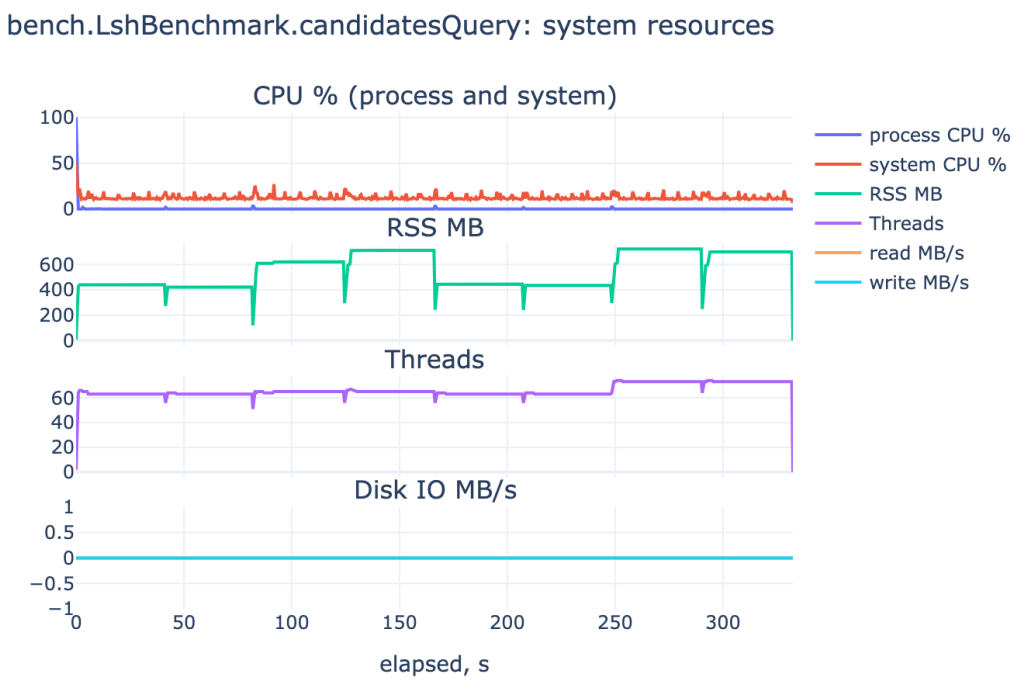

Figure 20: resources

Notes

- Получение кандидатов быстрое.

nearDuplicatesFullScan

- Operation type: `read/query`
- Parameters:
  - `docs`: 1500, 3000
  - `seed`: 42
  - `threshold`: 0.9
  - `wordsPerDoc`: 16

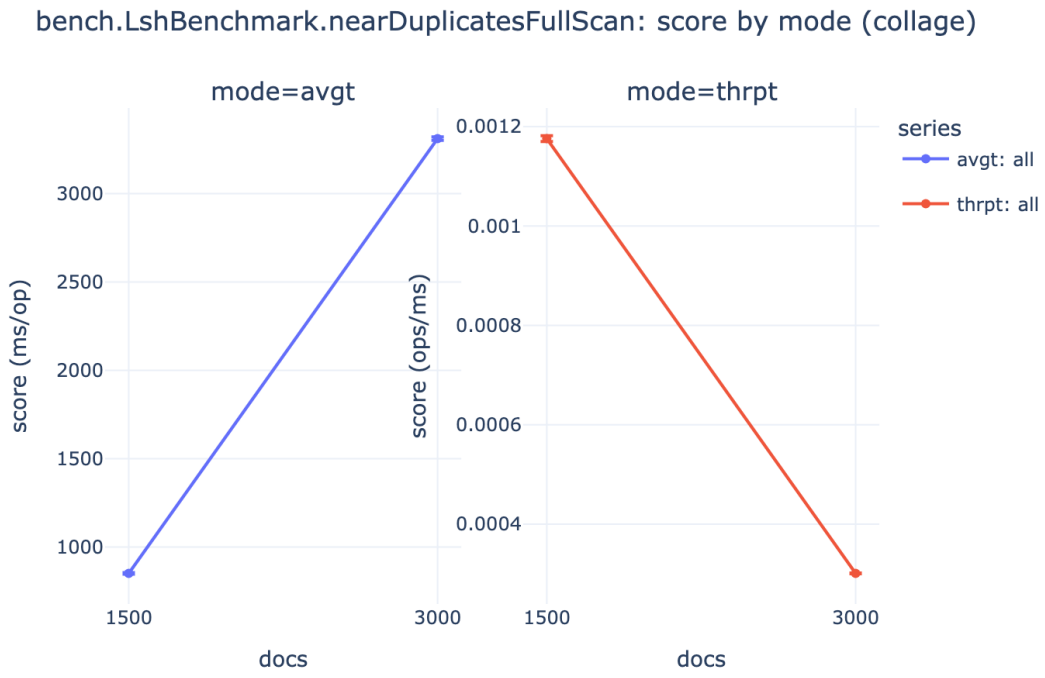

Figure 21: score

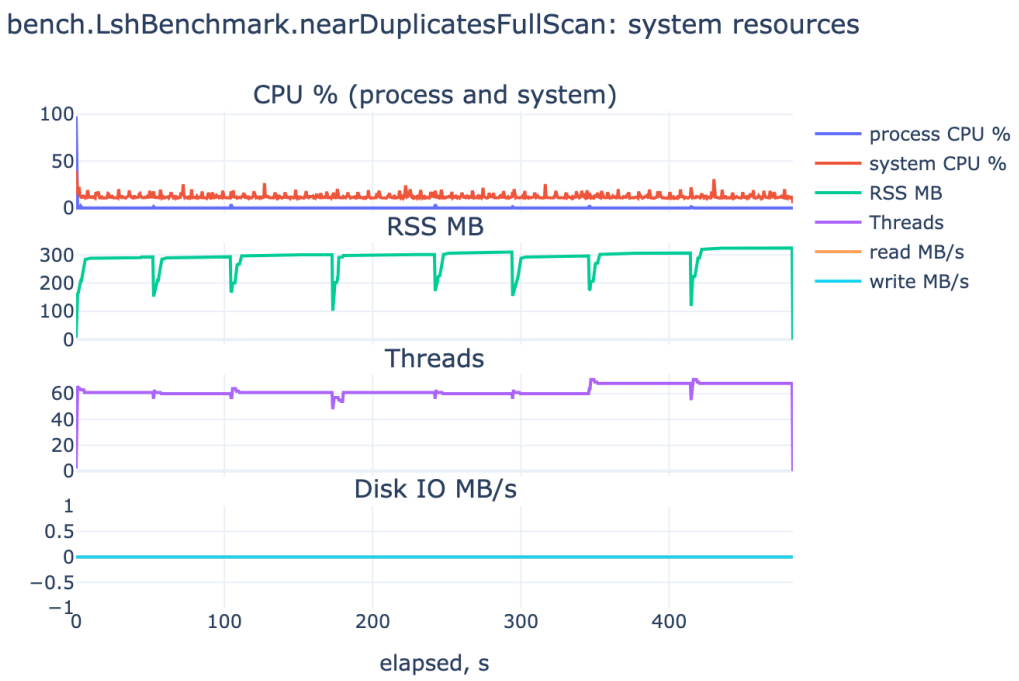

Figure 22: resources

Notes

- Full scan очень дорогой, но и нужен только для сравнения.

nearDuplicatesLsh

- Operation type: `read/query`
- Parameters:
  - `docs`: 6000, 12000
  - `seed`: 42
  - `threshold`: 0.9
  - `wordsPerDoc`: 16

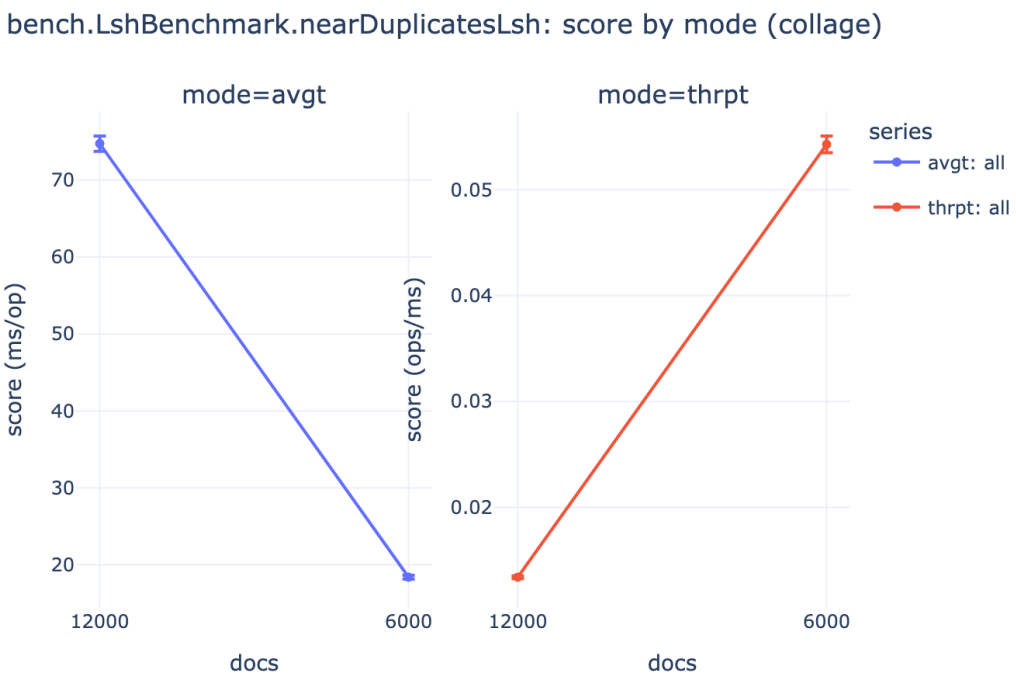

Figure 23: score

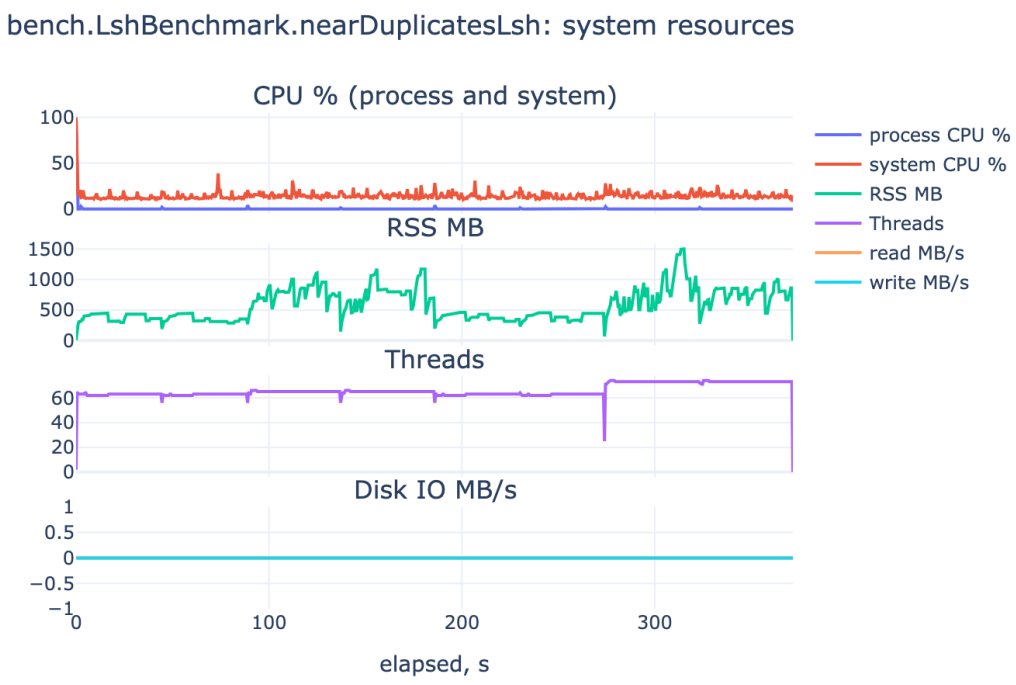

Figure 24: resources

Notes

- LSH-поиск существенно быстрее full scan.

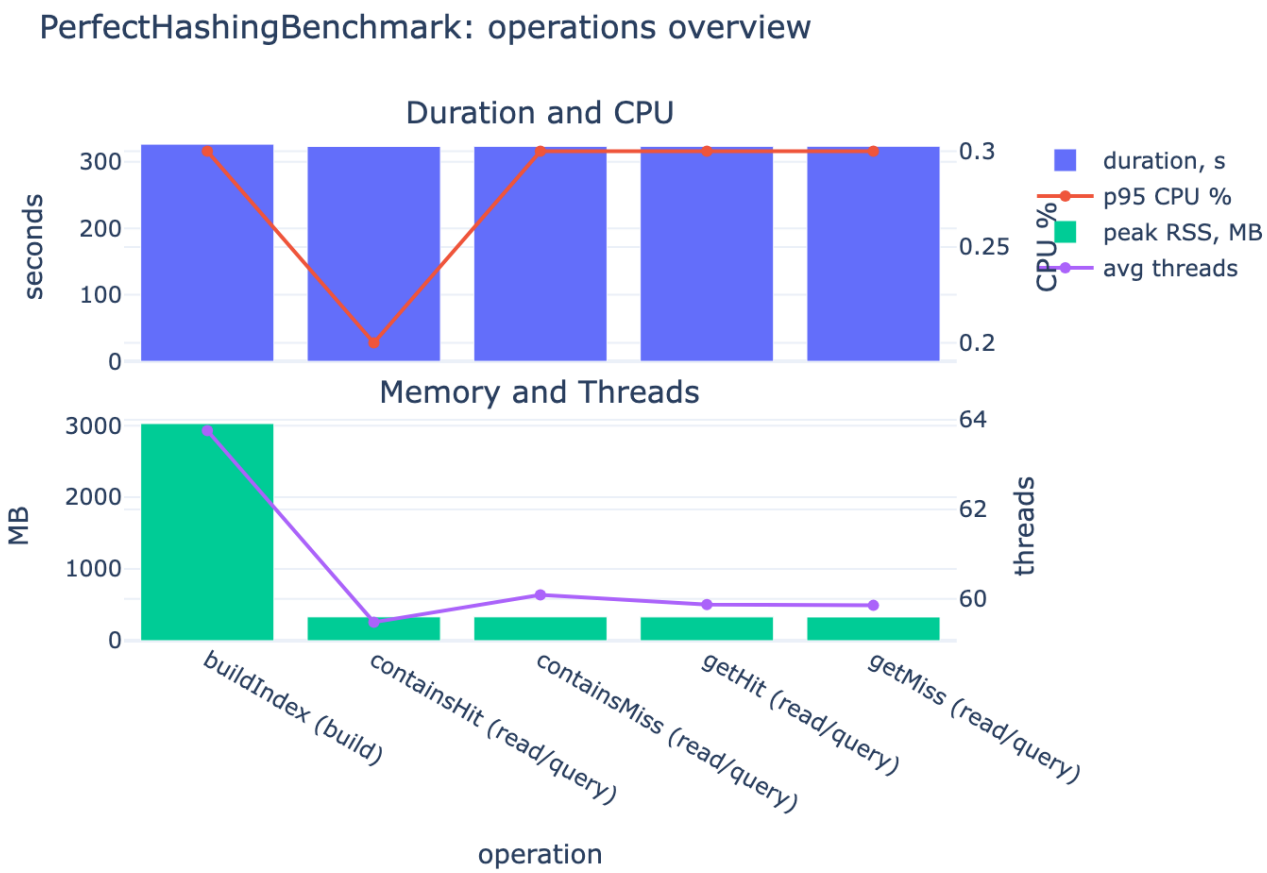

Figure 25: overview

operation	kind	duration_s	p95_cpu_%	peak_rss_mb	score_median
buildIndex	build	326.361	0.300	3027.984	3386.281
containsHit	read/query	323.100	0.200	331.641	15.627
containsMiss	read/query	323.184	0.300	331.859	14.979
getHit	read/query	323.207	0.300	330.938	13.926
getMiss	read/query	323.330	0.300	330.297	14.738

buildIndex

- Operation type: `build`
- Parameters:
  - n: 200000, 800000
  - `seed`: 42

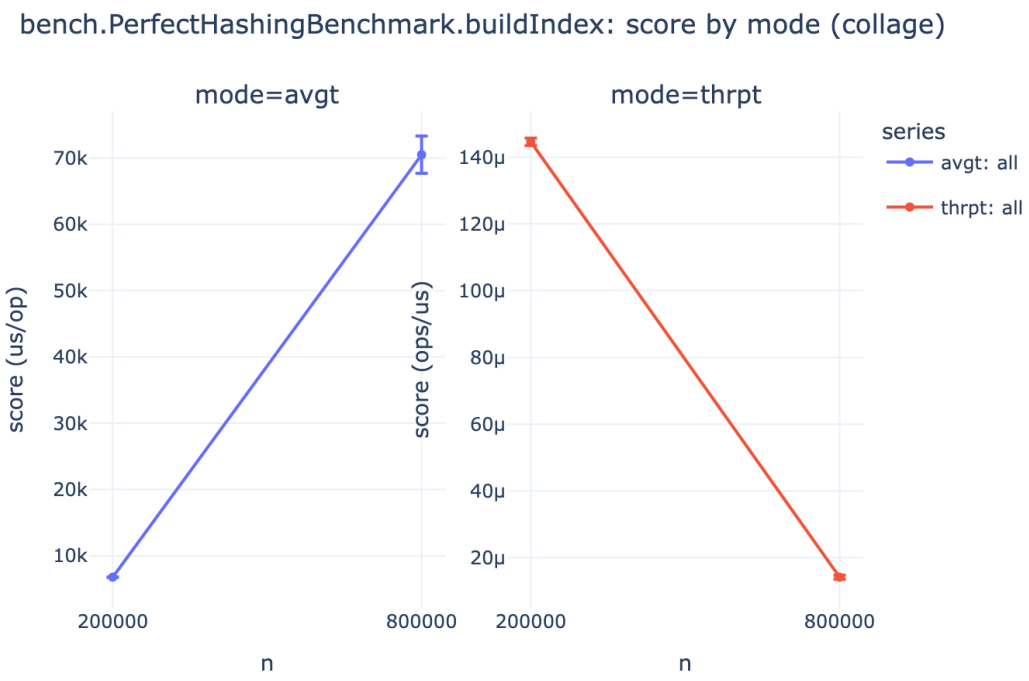

Figure 26: score

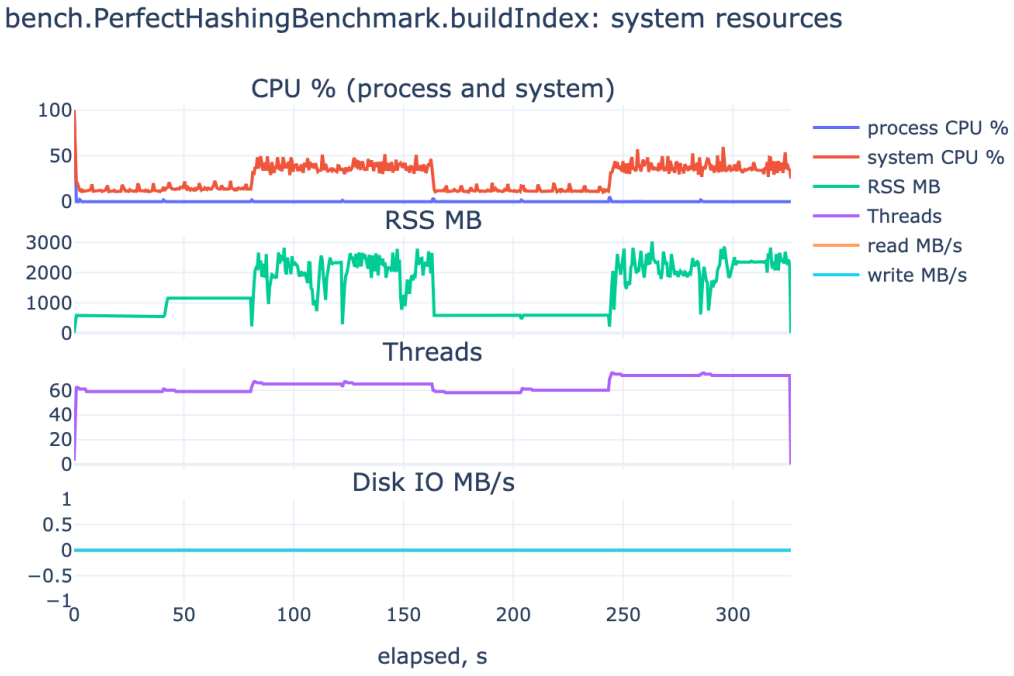

Figure 27: resources

Notes

- Индекс строится дорого и с большим разбросом: avgt median ~38.6 ms/op (в переводе из 38642 us/op).

- Фикс: сократить число повторений при подборе 2-го уровня, а также можно заранее сделать pre-size массивы 2-го уровня по размеру bucket^2.

containsHit

- Operation type: `read/query`
- Parameters:
  - n: 200000, 800000
  - seed: 42

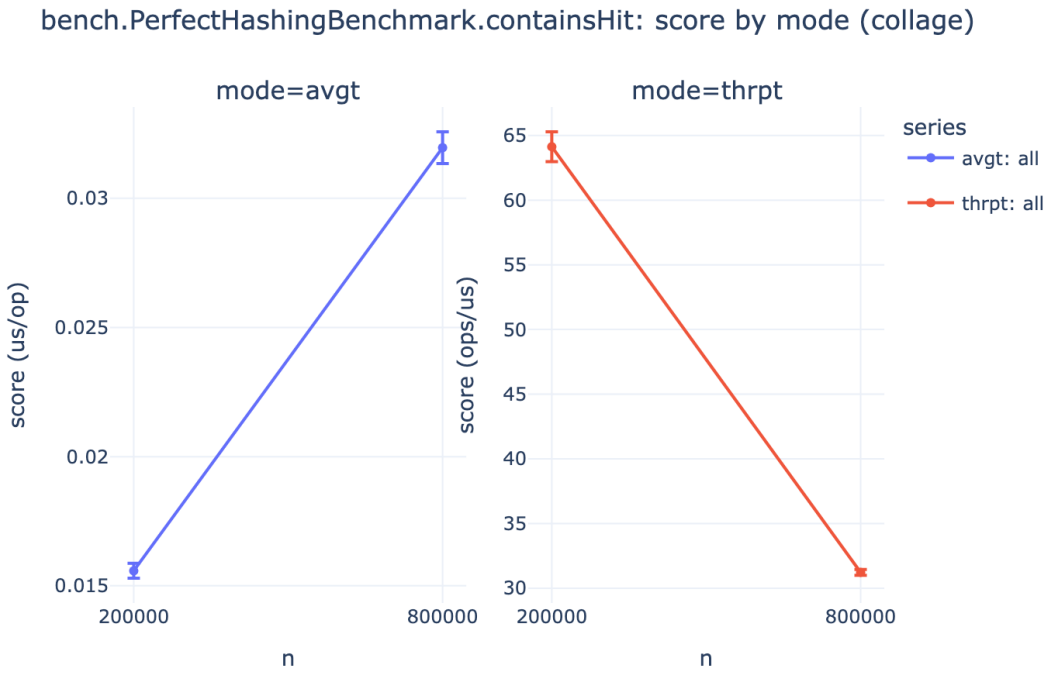

Figure 28: score

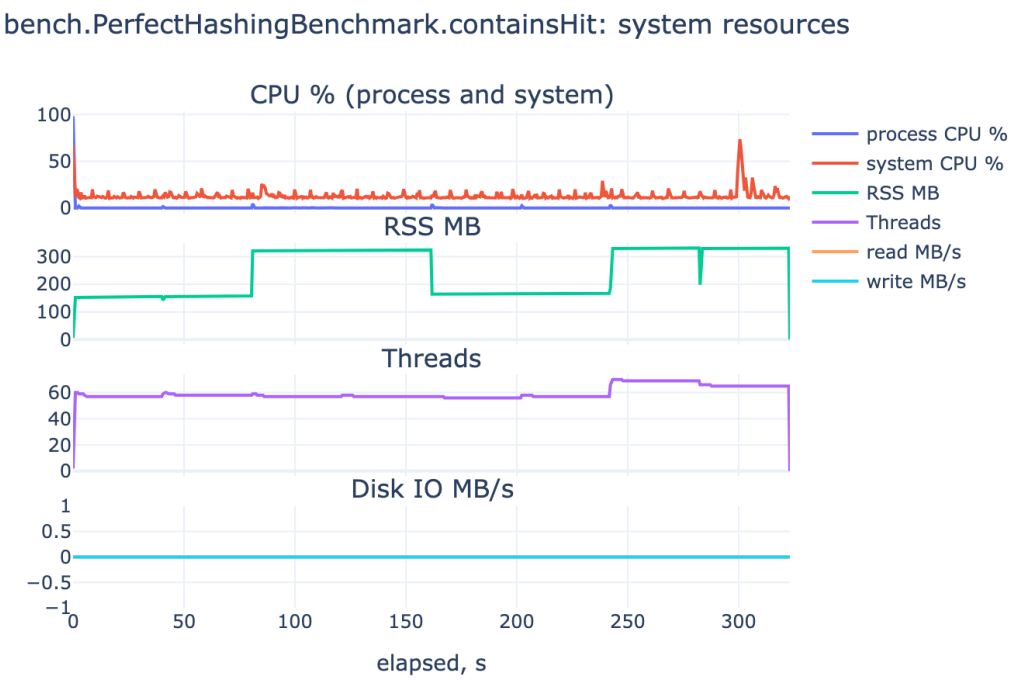

Figure 29: resources

Notes

- Просмотр быстрый и стабильный.

containsMiss

- Operation type: `read/query`
- Parameters:
  - n: 200000, 800000
  - `seed`: 42

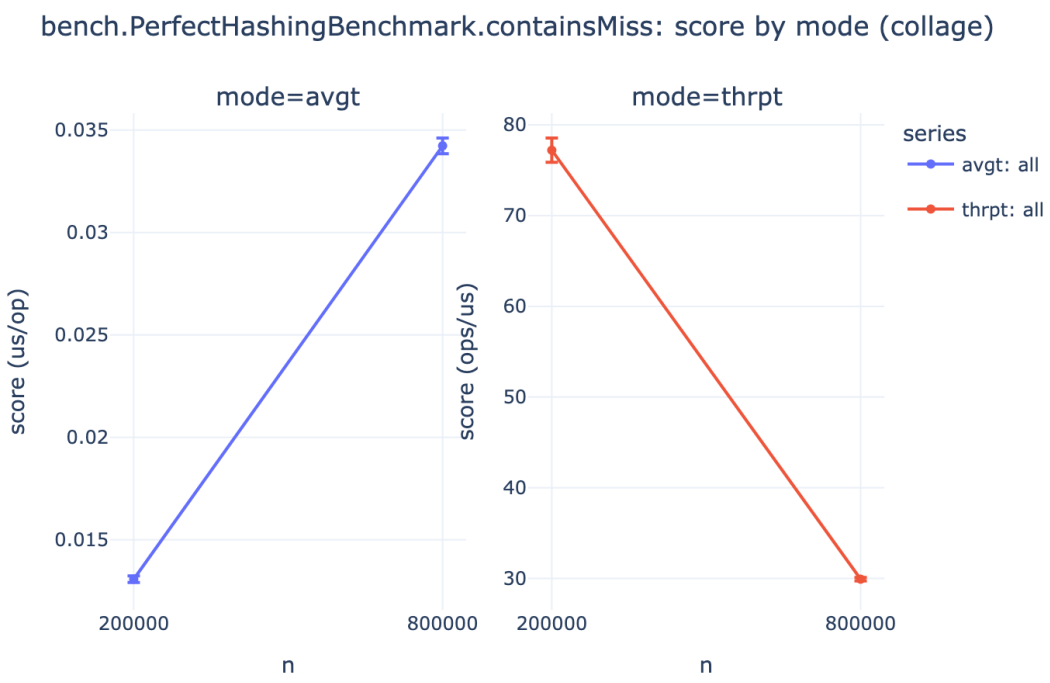

Figure 30: score

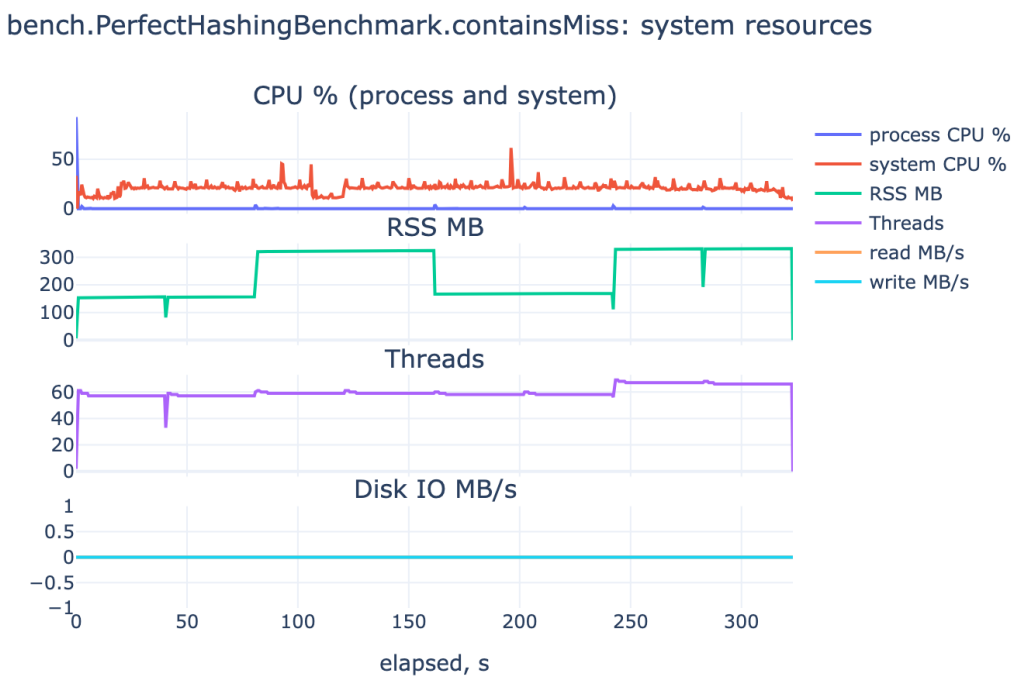

Figure 31: resources

Notes

- Miss сопоставим с hit.

getHit

- Operation type: `read/query`
- Parameters:
  - n: 200000, 800000
  - seed: 42

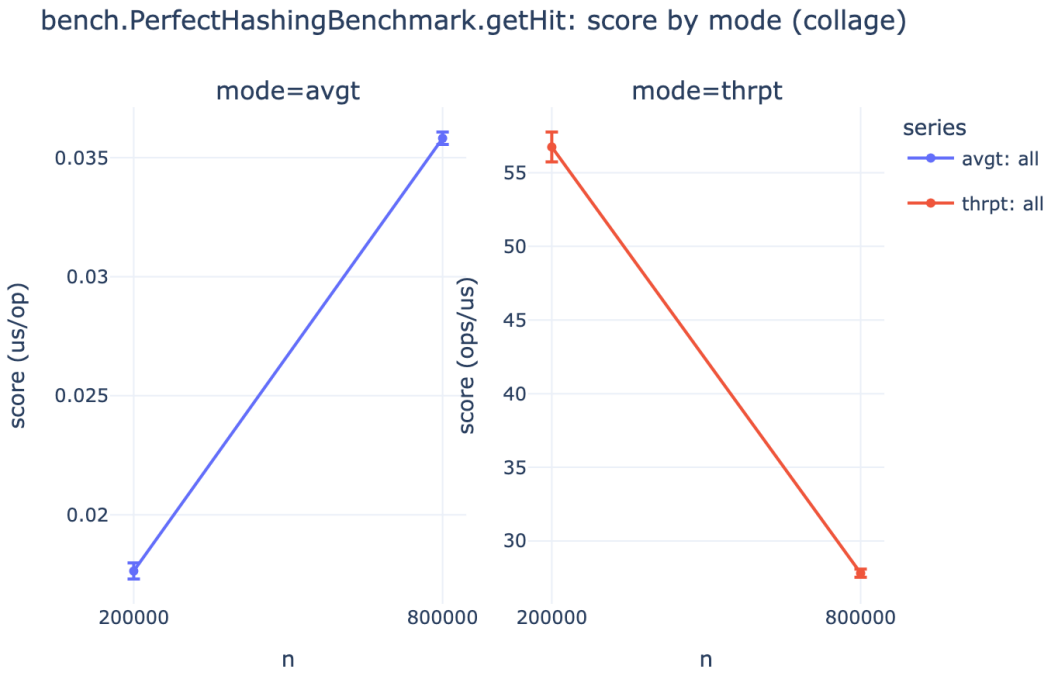

Figure 32: score

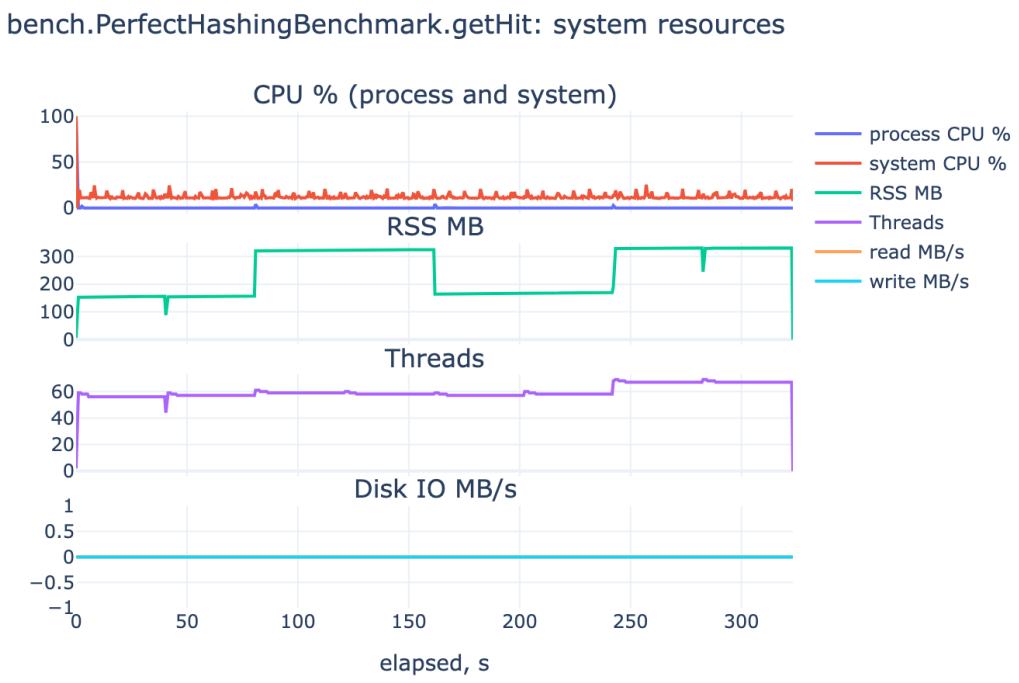

Figure 33: resources

Notes

- getHit чуть хуже containsHit из-за возврата значения.
- Фикс: хранить value в примитивном массиве/структуре.

getMiss

- Operation type: `read/query`
- Parameters:
  - n: 200000, 800000
  - `seed`: 42

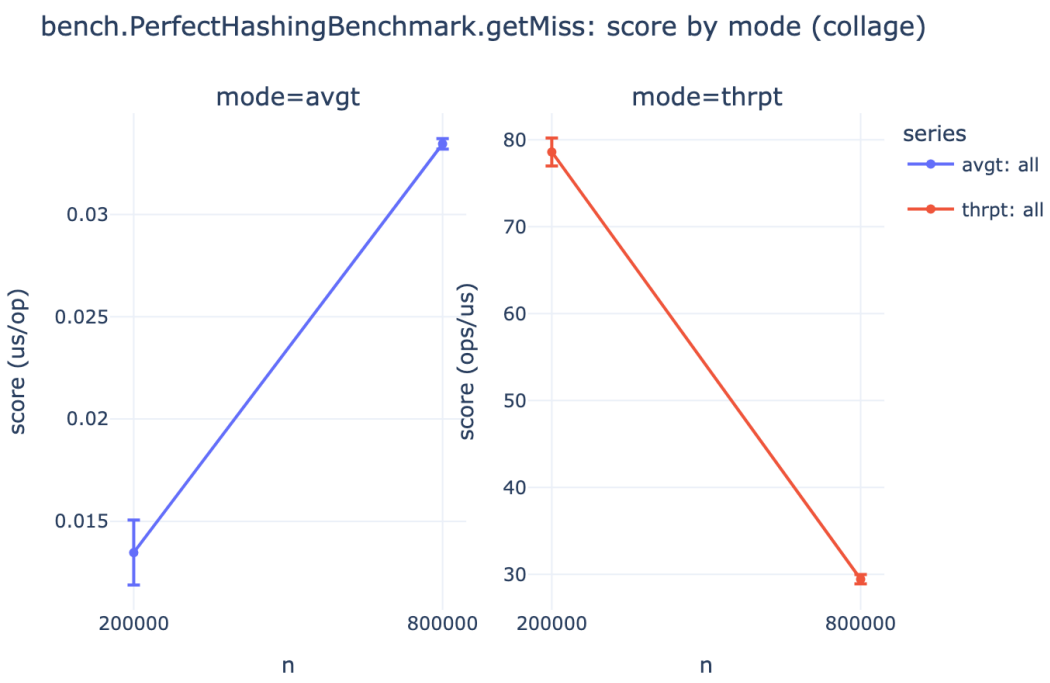

Figure 34: score

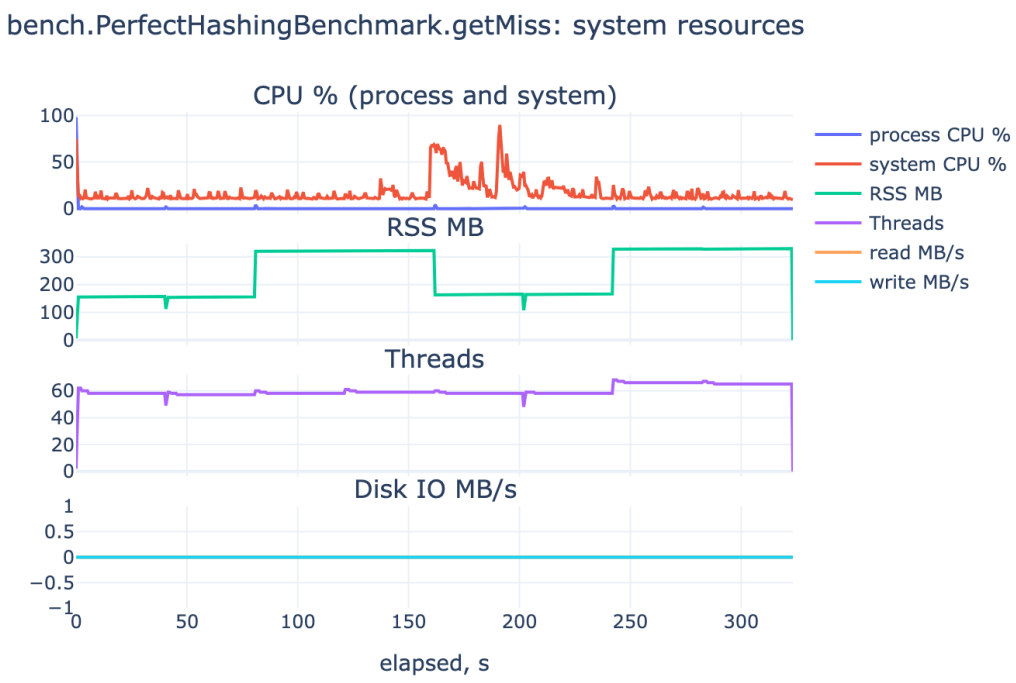

Figure 35: resources

Notes

- Фикс: унифицировать ветку miss между get/contains для одинакового поведения и меньшего кода.